UNIVERSIDAD DE SAN CARLOS DE GUATEMALA
CENTRO UNIVERSITARIO DE OCCIDENTE
DIVISIÓN DE CIENCIAS DE LA INGENIERÍA
MANEJO E IMPLEMENTACIÓN DE ARCHIVOS
SECCIÓN: “A”
SEGUNDO SEMESTRE 2026
ING. Bryan Misael Monzon Fuentes

MANUAL TÉCNICO DEL SISTEMA DE ALMACENAMIENTO EN LA NUBE

Helder Joel Juárez González 202331791

Quetzaltenango, 24 de Octubre de 2,026

Introducción 2
Objetivo 3
Arquitectura General del Sistema 4
Capa de Presentación (Frontend) 4
Capa de Lógica de Negocio (Backend) 4
Capa de Persistencia y Almacenamiento Binario 5
Seguridad y Control de Acceso (RBAC y ACL) 5
Integridad Transaccional (ACID) 5

Introducción

El presente Manual Técnico describe la arquitectura, los componentes y las decisiones de diseño de software implementadas en la construcción del sistema de gestión de archivos en la nube. La plataforma ha sido diseñada bajo un enfoque de alta disponibilidad y seguridad, permitiendo a los usuarios almacenar, organizar, compartir y auditar documentos digitales mediante un modelo jerárquico virtual. El sistema integra tecnologías modernas de desarrollo web, asegurando un procesamiento eficiente de los flujos binarios y una separación clara de responsabilidades entre la interfaz de usuario y el servidor central.

Objetivo

Establecer un documento formal de referencia que documente la arquitectura de software, los patrones de diseño, las topologías de despliegue y los esquemas de seguridad aplicados en el proyecto. Este manual sirve como guía técnica estandarizada para facilitar el mantenimiento, la depuración, la auditoría y la futura escalabilidad del sistema por parte de los equipos de ingeniería y administración.

Arquitectura General del Sistema

El sistema opera bajo una arquitectura distribuida de Cliente-Servidor (Client-Server), garantizando un bajo acoplamiento mediante la exposición de una API RESTful. Esta separación arquitectónica asegura que la capa de presentación y la capa de lógica de negocio puedan escalar, actualizarse y desplegarse de manera independiente. Toda la comunicación entre los nodos se realiza a través del protocolo HTTP/HTTPS, utilizando JSON (JavaScript Object Notation) como formato estándar de intercambio de mensajes.

Capa de Presentación (Frontend)

Desarrollada como una Single Page Application (SPA) utilizando el framework **Angular**.

Diseño Modular: Emplea una arquitectura basada en componentes *standalone*, reduciendo la dependencia de módulos globales y mejorando los tiempos de carga inicial mediante técnicas de Lazy Loading.

Reactividad y Manejo de Estado: Utiliza la librería **RxJS** (Observables y Subjects) para orquestar flujos de datos asíncronos. Esta reactividad es esencial para la actualización dinámica del DOM, permitiendo, por ejemplo, la recarga en tiempo real de la barra de almacenamiento global a través de un Subject que escucha las mutaciones del sistema.

Integración de Servicios: Las comunicaciones asíncronas con el backend se gestionan a través de la clase HttpClient. Se implementan interceptores para inyectar de forma transparente los encabezados de autorización (Tokens JWT) en cada solicitud saliente.

Capa de Lógica de Negocio (Backend)

Construida sobre el entorno de ejecución **Node.js** utilizando el framework de enrutamiento **Express.js**. El código del servidor implementa rigurosamente el patrón de diseño por Capas (Controller-Service-Repository) para garantizar el Principio de Responsabilidad Única (SRP):

Controladores (Controllers): Actúan como el punto de entrada de las peticiones HTTP. Extraen los parámetros, procesan el cuerpo de la solicitud (FormData para archivos binarios multiformes), delegan el procesamiento al servicio correspondiente y estructuran la respuesta HTTP con su respectivo código de estado estándar.

Servicios (Services): Encapsulan las reglas y la lógica de negocio pura. Ejecutan cálculos algorítmicos complejos (como la validación dinámica de cuotas de almacenamiento cruzada con el tamaño físico del archivo entrante), la resolución de conflictos de rutas y la verificación recursiva de los permisos anticibersecuestro.

Repositorios (Repositories): Proveen una capa de abstracción total sobre el Object Data Modeling (Mongoose). Centralizan las consultas a la base de datos, la ejecución de *Aggregation Pipelines* y la coordinación de sesiones transaccionales.

Capa de Persistencia y Almacenamiento Binario

La gestión persistente se delega al motor NoSQL **MongoDB**. Para optimizar el rendimiento y evitar la degradación del sistema, la base de datos se divide lógicamente en el tratamiento de datos:

Integración Mongoose-GridFS: El sistema almacena exclusivamente referencias ligeras (archivoGridFsId) y metadatos en las colecciones de BSON estándar. Los documentos físicos y flujos de carga pesados se procesan a través de tuberías (Readable.from y openUploadStream) que inyectan los fragmentos directamente en **GridFS**, optimizando el consumo de la memoria RAM del servidor.

Agregaciones Matemáticas: Las operaciones de cálculo de cuotas se resuelven nativamente en el motor de la base de datos a través de $aggregate. Esto permite sumar en bloque el campo peso de los archivos activos, delegando la carga computacional a MongoDB y liberando el hilo principal de Node.js.

Seguridad y Control de Acceso (RBAC y ACL)

Autenticación Stateless: Se utiliza el estándar JSON Web Tokens (JWT) para firmar y validar la identidad en cada transacción, eliminando la necesidad de mantener un estado de sesión en la memoria del servidor. Las credenciales se protegen aplicando un hash unidireccional con **bcrypt** (factor de costo 10).

Autorización Dual: Combina un Control de Acceso Basado en Roles (RBAC) para proteger endpoints críticos (reportes globales, modificación de límites) y Listas de Control de Acceso (ACL) mediante el subdocumento compartidoCon para administrar autorizaciones granulares (Lectura/Edición) en recursos individuales.

Blindaje Anticibersecuestro: Los servicios encargados del movimiento de archivos ejecutan un algoritmo de validación de herencia. Este mecanismo impide matemáticamente que un colaborador mueva un recurso ajeno hacia un directorio privado, garantizando que el propietario original jamás pierda visibilidad o control sobre sus datos.

Integridad Transaccional (ACID)

Dada la naturaleza distribuida entre las colecciones de metadatos y GridFS, el sistema garantiza la

coherencia operativa mediante el uso de **mongoose.startSession()**. Las rutinas de alta criticidad, como el reemplazo del contenido de un archivo o la migración recursiva de la ruta de una carpeta madre, se ejecutan dentro de bloques withTransaction. Ante cualquier excepción en la secuencia (fallo de red, disco lleno o violación de restricción), el bloque catch ejecuta un *rollback* lógico inmediato que cancela las escrituras en las colecciones y dispara rutinas de recolección de basura para expurgar los fragmentos binarios huérfanos generados en GridFS.